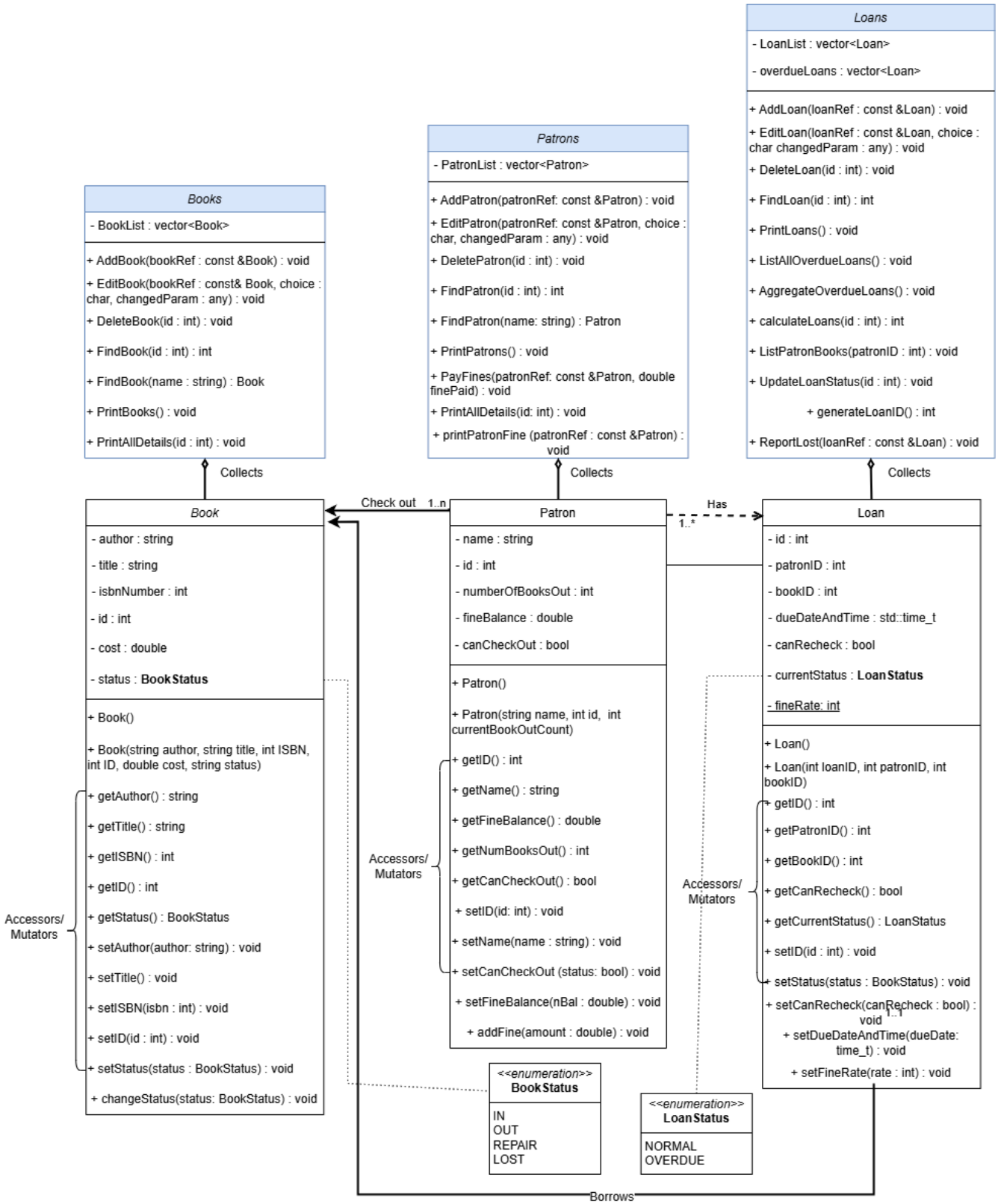
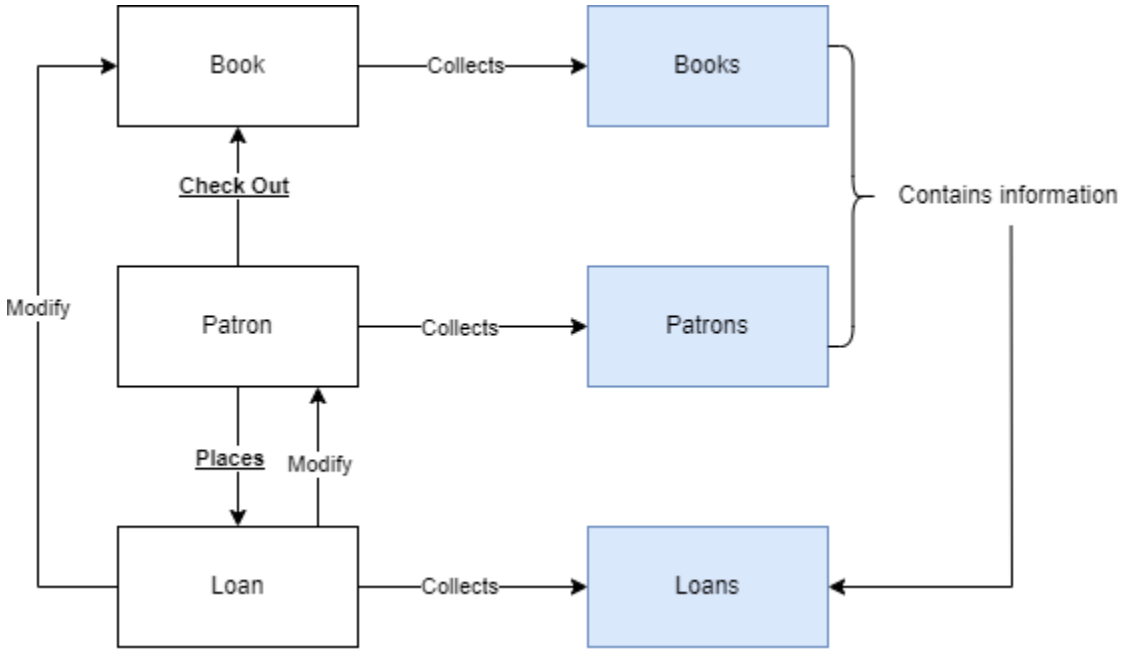
Kai (Sam) Gu
Prof. Mosquera
CSCE 1040.001
3 November 2024

HOMEWORK 3

Individual Class Diagrams (Modified)



Class Diagram Relationships (Unmodified)



Pseudocode Outline

Class: Books

addBook()

Prompt user for Author Prompt user for Title
Prompt user for ISBN_Number
Prompt user for Library ID
Prompt user for Cost
Create and populate **Book** Object via parameterized constructor
Add object to **Books** collection

findBook(int ID) = find by id

Prompt user for ID
Iterate through Book collection using for-loop
Check for ID that matches the user's given ID
Return index of **Book** in the **bookList** vector with the the matching ID

findBook(string name) = find by name

Prompt user for Name
Iterate through **Book** collection using for-loop
Check for Name that matches the user's given Name
Return index of the **Book** with the matching Name

editBook()

Prompt the user for **Book** ID
Use *findBook* to get the index of the **Book** with the user-specified ID
For the first parameter of *editBook()*, insert the memory reference of the **Book** object using the index of the **Book.**
Prompt the user for a char that represents the field/attribute to be changed and a string/int to replace that field to be changed (represented in param).
Check to make sure *findBook()* does not return -1 (i.e. book is in the **bookList** vector).
If *findBook()* does not return -1, call the mutator for the field identified in the char parameter to replace the attribute with the new user-prompted string or int. This function is called on the **Book** object referenced in the params.

> Ensure type safety by comparing the type of changedParam to type of source instance variable. Only proceed if type is the same.

deleteBook()

Prompt user for a Book ID
Use Method *findBook()* to get the index of the Book with the user-specified ID
If the index received from findBook() is not -1, make the **Books** Collection delete the index.

printBooks()

Iterate through the vector using a for-loop and print out all elements of the **Books** collection

printAllDetails()

Prompt the user for a Book ID
Use *findBook()* to get the index of the Book in the vector with the user's given ID
If *findBook()* does not return -1 (or that the Book does exist), Print out the Name, ID, Author, Title, ISBN_Number, and Cost of the element at the index returned by *findBook()*. (using accessors for each attribute)

Class: Loans

AddLoans() = check-in book

Prompt the user for a ***bookID*** and ***patronID***
Use *findBook(**bookID**)* to verify the **Book** is available
If *findBook(**bookID**)* returns -1, exit (book not found or already checked out)
Use *findPatron(**patronID**)*, then verify the **Patron** can check out books
If *findPatron(**patronID**)* returns -1, exit (patron not found)
Check if the **Patron** has fewer than 6 books checked out
If the **Patron** has 6 or more books, exit (cannot check out more books)
Create a new **Loan** object with the provided **Book ID**, **Patron ID**, and set the **Due Date** using <ctime> (std::time_t)
Set the **currentStatus** of the **Loan** to **NORMAL**
Add the **Loan** object to the **Loans** collection
Update the **Book's** status to **OUT**
Increment the **Patron's** current number of books out using the mutator function

findLoan(int ID) = find by ID

Prompt user for ID
Iterate through Loan collection using for-loop
Check for ID that matches the user's given ID
Return index of **Loan** in the **loanList** vector with the the matching ID

editLoan()

Prompt the user for **Loan** ID
Use *findLoan()* to get the index of the **Loan** with the user-specified ID
For the first parameter of *editLoan()*, insert the memory reference of the **Loan** object using the Loan index**.**
Prompt the user for a char that represents the field/attribute to be changed and a string/int to replace that field to be changed (represented in param).
Check to make sure *findLoan()* does not return -1 (i.e. loan is in the **loanList** vector)
If *findLoan()* does not return -1, call the mutator for the field identified in the char parameter to replace the attribute with the new user-prompted string or int. This function is called on the **Loan** object referenced in the params.

> Ensure type safety by comparing the type of changedParam to type of source instance variable. Only proceed if type is the same.

deleteLoan() = check out book

Prompt the user for a **Loan ID**
Use *findLoan(loanID)* to get the index of the **Loan** with the specified **Loan ID**
If *findLoan()* returns -1, exit (print "loan not found")
Retrieve the **Loan** object from the **Loans** collection using the index
Use *findBook(loan.bookID)* to get the index of the associated **Book**
Set the **Book's** status to **AVAILABLE** using the mutator function
Use *findPatron(loan.patronID)* to get the index of the associated **Patron**
Decrement the **Patron's** current number of books out using the mutator function
Check if the **Loan's currentStatus** is **OVERDUE**
If **OVERDUE**, calculate the fine using *calculateFine(loanIndex)*

> Add the calculated fine to the **Patron's** fine balance using the mutator function
>
> Delete **Loan** from **overdueLoans**

Delete the **Loan** from the **Loans** collection using the index.

printLoans()

Go through the **loanList** vector and print out all elements of the **Loans** collection.

aggregateOverdueLoans()

Iterate through the **Loans** collection using a for-loop.
For each **Loan**, check its **currentStatus**.
If the **currentStatus** is **OVERDUE**, add the **Loan** to the **overdueLoans** collection.

listAllOverdueLoans()

Call *aggregateOverdueLoans()* to get a collection of overdue **Loans**.
For each **Loan** in the **overdueLoans** collection:

Print the **Loan** details (Loan ID, Book ID, Patron ID, Due Date, and Fine).

updateLoanStatus()

Prompt the user for a **Loan Index**
Use *findLoan()* to get the index of the **Loan** in the collection
Make sure *findLoan()* does not return -1
Check if the **Loan** at the given index has a book that has not been held for over 10 days without renewal. If it has, make the instance variable **currentStatus** ***Overdue***.
Also check if the book can be renewed by looking at the instance variable **canRecheck**.

ReportLost()

From ***loanRef***, get the ***bookID*** and ***PatronID***

Use *findBook(**bookID**)* to get the associated **Book**
If *findBook(**bookID**)* doesn't return -1:

Set the **Book's** status to **LOST**

Use *findPatron(patronID)* to get the associated **Patron**
If *findPatron(patronID)* doesn't return -1:

Add **Book.cost** to **Patron's fineBalance**

Remove the loan from the loan collection and overdue collection if applicable.

Class: Patrons

addPatrons()

Prompt user for Name
Prompt user for ID
Prompt user for Current Number of Books Out
*Create and populate **Patron** object via parameterized constructor*
*Add object to **Patrons** collection*

findPatron(int ID) = find by id

Prompt user for ID

Linear search through **Patron** collection
Check for ID that matches the user's given ID
Return index of **Patron** in the **Patron** collection with the matching ID

findPatron(string name) = find by name

Prompt user for Name
Search through **Patron** collection
Check for Name that matches the user's given Name
Return index of the **Patron** with the matching Name

editPatron()

Prompt the user for **Patron** ID
Use *findPatron* to get the index of the **Patron** with the user-specified ID
For the param of *editPatron()*, insert the memory reference of the **Patron** object using the index of the **Patron**.
Prompt the user for a char that represents the field/attribute to be changed and a string/int to replace that field to be changed (represented in param).
Check to make sure *findPatron()* does not return -1 (i.e. Patron is in the **Patron** collection).
If *findPatron()* does not return -1, call the mutator for the field identified in the char parameter to replace the attribute with the new user-prompted string or int. This function is called on the **Patron** object referenced in the params.
Ensure type safety by comparing the type of changedParam to type of source instance variable. Only proceed if type is the same.

deletePatron()

Prompt user for a **Patron** ID
Use Method *findPatron()* to get the index of the **Patron** with the user-specified ID
If the index received from *findPatron()* is not -1, make the **Patron** collection delete the index.

printPatrons()

Iterate through the **patronList** vector using a for-loop and print out all elements of the **Patrons** collection

printAllDetails()

Prompt the user for a **Patron** ID
Use *findPatron()* to get the index of the **Patron** in the vector with the user's given ID
If *findPatron()* does not return -1 (or the **Patron** does exist), Print out the Name, ID, and Current Number of Books Checked Out of the element at the index returned by *findPatron()*.

payFines()

Prompt the user for **Patron** ID
Use *findPatron()* to get the index of the **Patron** with the user-specified ID

For the first parameter of *editLoan()*, insert the memory reference of the **Patron** object using the returned patron index.
If the second parameter (finePaid) is greater than 0 and less than or equal to the double returned by *getFineBalance()* for the **Patron** object reference, subtract the amount paid from the existing amount using the mutator *setFineBalance()*

Added Functions Pseudocode

Class: Books

Function: getBook(int index)

Check if the provided index is valid (not equal to -1).
If valid: Return the book located at BookList[index].
Else: Throw a runtime error with the message "Book not found."

Class: Loans

Function: getLoan(int index)

Display all current loans by calling printLoans().
Verify if the provided index is valid (not equal to -1).
If valid: Return the loan located at LoanList[index].
Else: Throw a runtime error with the message "Loan not found."

Function: generateLoanID() const

Initialize a variable maxID to 0.
Loop through each loan in LoanList:
If the loan's ID is greater than maxID:
Update maxID with this loan's ID.
After the loop, return maxID + 1 as the new unique Loan ID.

Function: calculateLoans(int id)

Find the index of the loan using findLoan(id).
Check if the loan exists (index is not -1).
If the loan exists: Retrieve the loan from LoanList using the found index.
Get the current time and store it in now.

Determine if the loan is overdue by comparing dueDateAndTime with now:
If overdue: Calculate daysOverdue by finding the difference between now and dueDateAndTime, then dividing by the number of seconds in a day.
Compute the fine by multiplying daysOverdue with fineRate.
Display a message indicating the loan ID, days overdue, and the fine amount.
Else: Display a message stating that the loan is not overdue.
Else: Display a message saying "Loan not found."

Class: Patrons

Function: getPatron(int index)

Check if the provided index is valid (not equal to -1).
If valid: Return the patron located at PatronList[index].
Else: Throw a runtime error with the message "Patron not found."

Function: printPatronFine(const Patron& patronRef)

Display the patron's name followed by their current fine balance in the format: "[Patron Name]'s fine balance: $[Fine Amount]".
Return the patron's fine balance.

Rest of the added functions are Getters/Setters, hence not applicable.

Report (for HW 2 Design)

For this project, I first designed a basic outline of its functionality but with most of its required data members as described on the project guidelines. After that, I realized that we should follow the notations of a UML class diagram, including the denotation of public/private instance variables within the class, denotation of static, and inter-class relationships. Following the syntax, I constructed a more detailed and refined version of the class while declaring the entire function signature, ensuring consistency across various functions that will be called within each other. I also focused on using memory references when an object needs to be called to ensure performance, and also in-class relationships by keeping variables private and thus protected. I also utilized ***enum*** as some variables require between 2 or 4 states.

Through this project, I gained a deeper understanding of UML and system design in general and its importance in structuring efficient class and program designs. In future projects, I will leverage this knowledge to plan inter-class relationships more effectively, minimizing the need for redrafting and doing repetitive work, making my development process smoother.